# Experience-aware staffing for Float

Oct 17, 2026 · Vyom Bhushan

Smart assign suggests who should fill an open role by matching job titles and checking who is free. It doesn't look at what people have actually worked on (Float's help article). I built a version that does, and tested both on an agency loaded into Float. For small teams the two are about the same. From 100 people up, the version that knows people's past work picks the right person first more often.

## The problem

When a manager opens an unfilled role, Smart assign suggests three people. It picks them on two things: does their job title match, and are they free on those dates. That is what Float's help article says, and it is what I saw in a live account: for a Senior Designer role, the same three people came up whether the project was for a bank, a retailer or a hospital.

In a 50-person agency that barely matters. There are usually only two or three Senior Designers, so "a free Senior Designer" is nearly the answer already. In a 300-person agency there might be fifteen. Then the useful question is not who is free, but who has done this kind of work for this kind of client before. Float has said this is where it wants to go: the Smart assign launch post promises to add "the relationship between your people, their skills, and the work they deliver".

## What I built

A version of the suggestion that looks at the same two things plus four more: whether the person has worked for this client before, whether they have done similar projects, whether their skills match the brief, and whether they have worked with the people already on the team. It is a small ranking model with seven inputs, fitted on the agency's own staffing history, so it learns how much each of those matters for that agency. No LLM is involved in the ranking.

## How I tested it

There is no public Float data, so I made some. I invented a 54-person agency with 18 months of projects and 459 staffing decisions, and loaded it into a Float trial through the public API (about 960 calls, within Float's rate limits, resumable if interrupted), so Float's

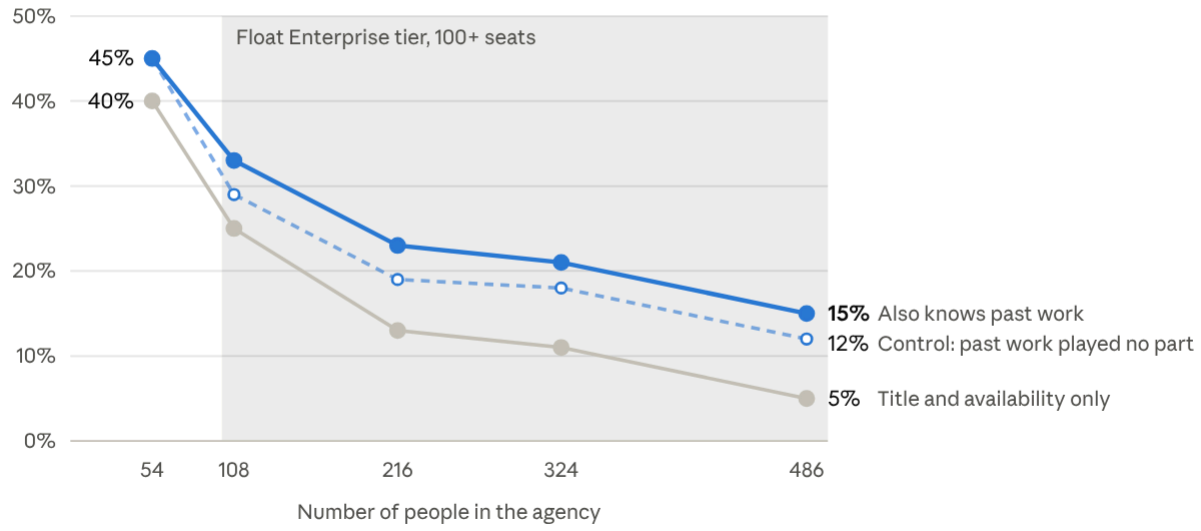
own Smart assign could run on it. Everything downstream reads it back out through the same API, read-only. A dbt project rebuilds the inputs in SQL with tests, including one that no input uses anything that happened after the decision it feeds.

The test is simple. Take each past staffing decision, hide who got the job, and ask both versions to suggest someone using only what was known at the time. Then check: was the first suggestion the person who actually got the job? The chart shows how often it was.

To see what happens as teams grow, I generated bigger agencies the same way, up to 486 people, and ran the same test on them outside Float.

**Knowing people's past work helps once a team passes about 100 people**

How often the first suggestion was the person who actually got the job




simulated agencies of each size, 3 versions each, averaged

## What I found

- **At 54 people, almost no difference.** Both versions get the first pick right about 4 times in 10.
- **From about 100 people, past work starts to help.** The gap is 3 to 5 points on the first suggestion and holds at every size I tried above 100. Across the top three it is smaller, 2 to 6 points, because the right person was often already in Float's three. Not dramatic, but it sits in Float's Enterprise tier, where the title-only approach is also at its weakest.
- **The control line is there to keep me honest.** I also made agencies where past work played no part in who got staffed. On those, my version learns that history does not matter and ignores it. So the gap in the real runs is not something I built in by accident.

## What this does not show

The agency is invented and the staffing rules are mine. I staffed it with a simple rule that favoured people who had worked for the client before, had done similar work, got on with the producer and were free, plus a dose of randomness and some "whoever is free" picks. So the result says how the two approaches behave as teams grow, not what Float's customers would see. The real test is running the same thing on real staffing history. It needs nothing exotic: allocations, project roles and time off, which are already in your warehouse.

## One more thing

Smart assign's suggestions live only inside the Float app. You cannot get them through the API or through Float MCP, so you cannot ask Claude "who should take this role?" I wrapped my version in a small MCP server so you can, and it tells you why it picked each person.

Everything is in the repo: the code that loads Float, the test, the 20 live Smart assign results, and a dbt version of the data pipeline with tests.

One question: is this close to what NextGen is about, or does it sit elsewhere?